
Basic Kuberentes commands Part2

Rohit Surwade

CONTENT

[To start Minikube](#)

[ANother way to delete pods\(Delete the manifest\)](#)

[A manifest to create pod with Environment Variable](#)

[Enter the container to check the Environment variables](#)

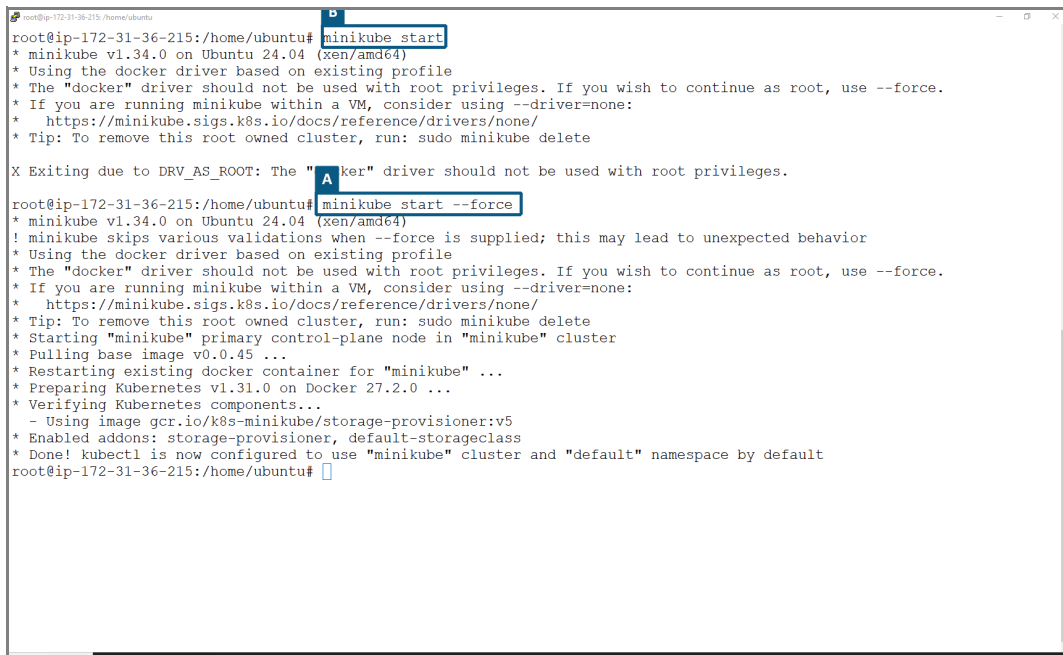
[To check the ENV](#)

[To delete the Pod](#)

[An example of Manifest with port exposing](#)

[How to ping on the Specified ports](#)

To start Minikube



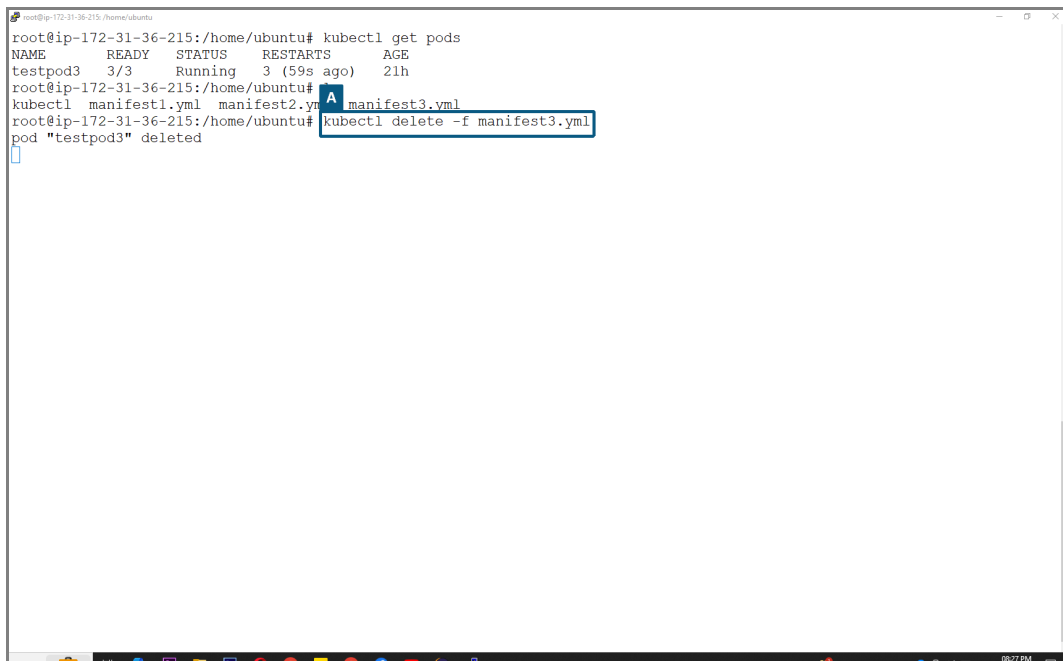
```
root@ip-172-31-36-215:/home/ubuntu# minikube start
* minikube v1.34.0 on Ubuntu 24.04 (xen/amd64)
* Using the docker driver based on existing profile
* The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
* If you are running minikube within a VM, consider using --driver=none:
  https://minikube.sigs.k8s.io/docs/reference/drivers/none/
* Tip: To remove this root owned cluster, run: sudo minikube delete

X Exiting due to DRV_AS_ROOT: The "docker" driver should not be used with root privileges.

root@ip-172-31-36-215:/home/ubuntu# minikube start --force
* minikube v1.34.0 on Ubuntu 24.04 (xen/amd64)
! minikube skips various validations when --force is supplied; this may lead to unexpected behavior
* Using the docker driver based on existing profile
* The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
* If you are running minikube within a VM, consider using --driver=none:
  https://minikube.sigs.k8s.io/docs/reference/drivers/none/
* Tip: To remove this root owned cluster, run: sudo minikube delete
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.45 ...
* Restarting existing docker container for "minikube" ...
* Preparing Kubernetes v1.31.0 on Docker 27.2.0 ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
root@ip-172-31-36-215:/home/ubuntu#
```

1.
minikube start
minikube start --force

ANother way to delete pods(Delete the manifest)



```
root@ip-172-31-36-215:/home/ubuntu# kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
testpod3   3/3     Running   3 (59s ago)  21h
root@ip-172-31-36-215:/home/ubuntu# kubectl manifest1.yml manifest2.yml manifest3.yml
root@ip-172-31-36-215:/home/ubuntu# kubectl delete -f manifest3.yml
pod "testpod3" deleted
root@ip-172-31-36-215:/home/ubuntu#
```

2.
kubectl delete -f manifest_name.yml

A manifest to create pod with Environment Variable

```
root@ip-172-31-36-215: /home/ubuntu
apiVersion: v1
kind: Pod
metadata:
  name: leo
spec:
  containers:
  - name: c00
    image: ubuntu
    command: ["/bin/bash", "-c", "while true; do echo Titanic; sleep 5; done"]
    env:
      # List of environment variables to be used inside the pod
      - name: MYNAME
        value: "Rohit Khanna"
```

3.

POD ENVIRONMENT VARIABLES

(This manifest defines a Kubernetes Pod named leo that runs a single container named c00 using the Ubuntu image.

The container executes a command that continuously prints "Titanic" every 5 seconds.

Additionally, it sets an environment variable MYNAME with the value "Rohit Khanna",

which can be accessed by any processes running inside the container.

the environment variable section of your Kubernetes Pod manifest allows you to define key-value pairs that can be accessed

by the application running inside the container. This feature enhances the flexibility, security, and maintainability of your applications,

making it a best practice in containerized environments. By using environment variables,

you can easily configure your applications without hardcoding values, leading to cleaner and more manageable code.)

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
name: leo
```

```
spec:
```

```
containers:
```

```
- name: c00
```

```
image: ubuntu
```

```
command: ["/bin/bash", "-c", "while true; do echo Titanic; sleep 5; done"]
```

```
env: # List of environment variables to be used inside the pod
```

```
- name: MYNAME
```

```
value: "Rohit Khanna"
```

Enter the container to check the Environment variables

```

root@leo: /
kubect1 manifest1.yml manifest2.yml manifest3.yml
root@ip-172-31-36-215:/home/ubuntu#
root@ip-172-31-36-215:/home/ubuntu# vi manifest4.yml
root@ip-172-31-36-215:/home/ubuntu# A kubectl apply -f manifest4.yml
pod/leo created
root@ip-172-31-36-215:/home/ubuntu# B kubectl exec leo -it -c c00 -- bin/bash
root@leo:/# C env
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
MYNAME=Rohit Khanna
HOSTNAME=leo
PWD=/
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33:01:cd=40;33:01:or=40;31:01:mi=00:su=37;41:sg=3
w=30;42:ow=34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01;31:*.taz=01;31:*.lha=01;31:*.lzh=01;31:*.
lzma=01;31:*.tlz=01;31:*.txz=01;31:*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31:*.lrz=01;31:*.lz=0
1;31:*.xz=01;31:*.zst=01;31:*.tztst=01;31:*.bz2=01;31:*.bz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;
31:*.war=01;31:*.ear=01;31:*.sar=01;31:*.rar=01;31:*.alz=01;31:*.ace=01;31:*.zoo=01;31:*.cpio=01;31:*.7z=01;31:*.rz=01;31
:*.wim=01;31:*.swm=01;31:*.dwm=01;31:*.esd=01;31:*.avif=01;35:*.jpg=01;35:*.jpeg=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=0
1;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=0
1;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.webp=01;35:*.og
4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:*.flc
=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01
;35:*.aac=00;36:*.au=00;36:*.flac=00;36:*.m4a=00;36:*.mid=00;36:*.midi=00;36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00
36:*.wav=00;36:*.oga=00;36:*.opus=00;36:*.spx=00;36:*.xspf=00;36:*.~#00;90:*.bak=00;90:*.crdownload=00;90:*.dpgk-
.dpkg-new=00;90:*.dpkg-old=00;90:*.dpkg-tmp=00;90:*.old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.rpmnew=00;90:*.rpmp
rpmsave=00;90:*.swp=00;90:*.tmp=00;90:*.ucf-dist=00;90:*.ucf-new=00;90:*.ucf-old=00;90:
TERM=xterm
SHLV=L1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PORT=443
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
_/usr/bin/env

```

4.

```
kubectl exec pod_name -it -c container_name -- bin/bash
```

To check the ENV

```

root@leo: /
root@leo:/# A echo $MYNAME
Rohit Khanna
root@leo:/#

```

5.

To delete the Pod

```
root@leo:/# echo $MYNAME
Rohit Khanna
root@leo:/# exit
exit
root@ip-172-31-36-215:/home/ubuntu# ls
kubect1 manifest1.yml manifest2.yml manifest3.yml manifest4.yml
root@ip-172-31-36-215:/home/ubuntu# kubect1 get pods
NAME      READY   STATUS    RESTARTS   AGE
leo        1/1     Running   0           14s
root@ip-172-31-36-215:/home/ubuntu# kubect1 delete pod leo
pod "leo" deleted
```

6.

```
kubect1 delete pod pod_name
```

An example of Manifest with port exposing

```
root@ip-172-31-36-215: /home/ubuntu
kind: Pod
apiVersion: v1
metadata:
  name: testpod4
spec:
  containers:
  - name: c00
    image: httpd
    ports:
    - containerPort: 80

-- INSERT --
```

7.

POD WITH PORTS

(This manifest defines a Kubernetes Pod named testpod4 that runs a single container based on the httpd (Apache HTTP Server) image.

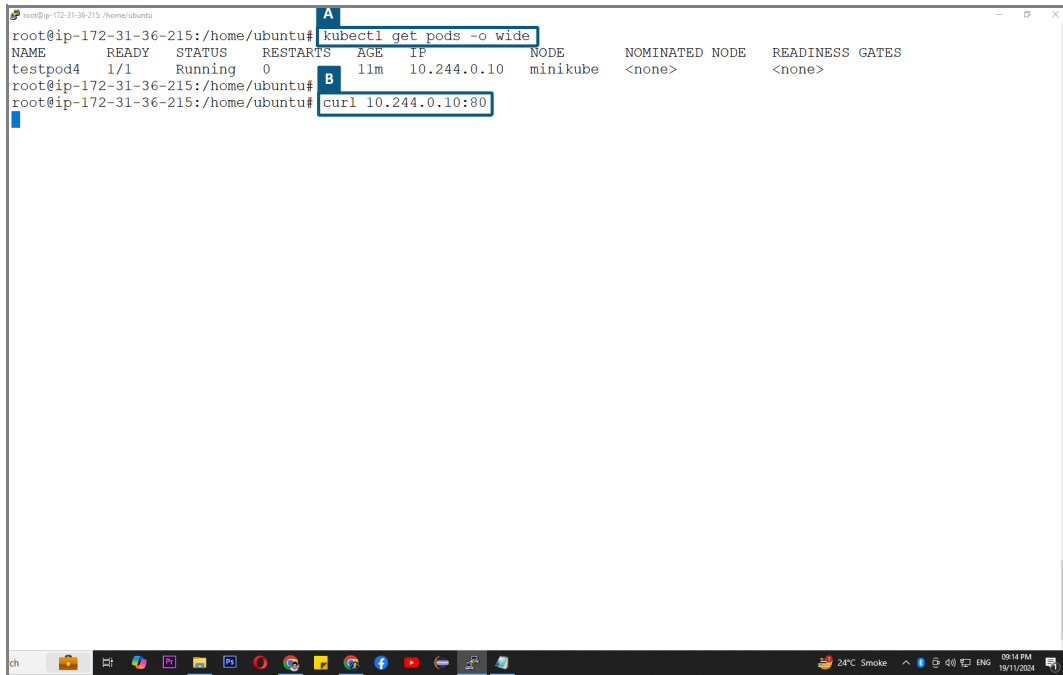
The container is configured to expose port 80, which is commonly used for serving web traffic.

This Pod can be used to run a web server, and you can access it through port 80 once the Pod is deployed and running in your Kubernetes cluster.

If you want to expose this Pod to external traffic, you would typically create a Service resource that targets this Pod.)

```
kind: Pod
apiVersion: v1
metadata:
  name: testpod4
spec:
  containers:
  - name: c00
    image: httpd
    ports:
    - containerPort: 80
```

How to ping on the Specified ports



The screenshot shows a terminal window with the following content:

```
root@ip-172-31-36-215:/home/ubuntu# A kubect1 get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
testpod4  1/1     Running   0           11m   10.244.0.10   minikube  <none>            <none>
root@ip-172-31-36-215:/home/ubuntu# B curl 10.244.0.10:80
```

Below the terminal screenshot, the following text is displayed:

8.

kubect1 get pods -o wide (to get detailed info about pods such as IP address)

curl POD_IP:PORT_NO